Problem statement

What JDK collections and streams leave on the heap

00

The problem in one slide

JDK collections and streams spend heap on objects the code never mentions.

- Boxed `Integer`s and per-entry nodes inflate what a collection holds.
- Every stream call builds a pipeline of objects and drops it.
- How much garbage depends on the JIT, the `Integer` cache and the headers.
- Fat-free (non-capturing) lambdas and plain loops leave far less behind.

Agenda

What stays on the heap, what churns through it, and why both matter.

#	Section	In one line
01	Baseline and volatile heap	Two heaps in one: what stays and what churns
02	jmap in the wild	Class histograms of IntelliJ IDEA and SonarQube for IDE
03	Movers: the baseline	What JDK collections hold, measured with JOL
04	Shakers: the volatile part	What every stream call throws away, measured with JMH
05	The problem, restated	The problem statement in four lines